Chapter 21 : Reinforcement Learning

1. Reinforcement Learning Setting

- An agent interacts with an environment over time.
- Receives a numeric reward for each action.
- Goal: maximize expected cumulative reward.
- In this chapter we study passive reinforcement learning: the policy is fixed, we only evaluate it.

Key Idea: Learn the utility (expected return) of states from experience while following a fixed policy.

2. Fixed Policy (Passive Reinforcement Learning)

- A policy $\pi(a|s)$ is given and fixed.
- The agent follows the policy and observes a sequence of states, actions and rewards $\langle s_0, a_0, r_0, s_1, a_1, r_1, \ldots \rangle$.
- Learn the utility $U_\pi(s)$ = expected return starting from state s and following policy π.
- No need to choose actions (that comes later in active RL).

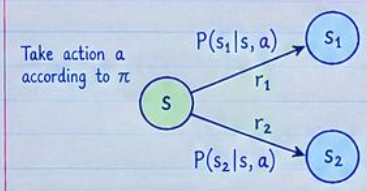
Definition: $$U_\pi(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \,\middle|\, s_0 = s, \text{ policy } \pi\right]$$

γ : discount factor, $0 \le \gamma < 1$.

3. Bellman Utility Intuition

- The utility of a state depends on the utilities of its successor states.
- Under a fixed policy π, the next state is a random variable determined by π and the environment model P.
- Bellman equation for a fixed policy:

$$U_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma U_\pi(s')\right]$$



$$U_\pi(s) = E\left[r + \gamma U_\pi(s')\right] = \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma U_\pi(s')\right]$$

4. Direct Utility Estimation (Monte Carlo)

- Estimate $U_\pi(s)$ directly from complete episodes (no model).
- For each visit to state s, compute the actual return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$ and average.
- As the number of visits increases, the average converges to $U_\pi(s)$ (law of large numbers).

Monte Carlo update (every-visit):

$$U(s) \leftarrow (1-\alpha)U(s) + \alpha G_t$$

Return from time t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Pros:
- Simple and model-free.
- Unbiased estimates (in the limit).

Cons:
- Must wait until the end of an episode.
- High variance.

5. Adaptive Dynamic Programming

- Use the Bellman equation as an update rule, but without knowing the model P and R.
- Estimate the expected value of the RHS from sampled transitions while following the policy.

Sample-based backup:

$$U(s) \leftarrow (1-\alpha)U(s) + \alpha\left[R + \gamma U(s')\right]$$

where s' is the next state observed after taking action a under the fixed policy π.

This is a model-free version of iterative evaluation (dynamic programming using sampled experience).

6. Temporal-Difference Learning

- Update the estimate after each step, without waiting for the end of the episode.
- Uses a bootstrapped estimate $\gamma U(s')$ for the remaining future (hence lower variance than Monte Carlo).

TD(0) update for state s at time t:

$$U(s) \leftarrow (1-\alpha)U(s) + \alpha\left[R_{t+1} + \gamma U(s')\right] = U(s) + \alpha\underbrace{\left[R_{t+1} + \gamma U(s') - U(s)\right]}_{\text{temporal-difference (TD) error } \delta_t}$$

TD(0) is the most basic TD method. It is a special case of more general n-step and TD(λ) methods.

7. Estimating U(s) in Practice

- Maintain a table of utility estimates for each state.
- Initialize $U(s)$ to 0 (or optimistic/pessimistic values).
- For each observed transition (s, r, s') following policy π, update using a TD(0) or adaptive DP rule.

TD(0) for a single transition:

1. Observe transition $s \xrightarrow{(a,r)} s'$ (a chosen by π).
2. Compute $\delta = r + \gamma U(s') - U(s)$.
3. Update $U(s) \leftarrow U(s) + \alpha\,\delta$.

With enough experience, $U(s)$ converges to $U_\pi(s)$ (for stationary environments and diminishing α).

8. Model-Based vs. Model-Free Intuition

Model-Based Approach
- Learn or know the model $P(s'|s,a)$ and rewards $R(s,a,s')$.
- Use the Bellman equation and iterative policy evaluation.
- Makes full use of the model.
- Generalizes to unvisited states.
- But requires learning/maintaining a transition model.

Model-Free Approach
- Do not learn the model explicitly.
- Estimate $U(s)$ directly from sampled experience (e.g., TD).
- Simpler and often more scalable.
- Works even when the model is unknown or very large.
- May require many visits and does not easily generalize.

9. Summary Comparison (Passive RL)

Method	Model needed?	Update target	When to update?	Variance
Monte Carlo	No	Actual return G_t	End of episode	High
Adaptive DP	No	$R + \gamma U(s')$	Each step	Lower
TD(0)	No	$R + \gamma U(s')$	Each step	Lower
Model-based DP (if P known)	Yes	Expectation over P and $U(s')$	Iterate over states	Lowest

> "Reinforcement learning allows an agent to learn what is good from experience." — AIMA (Ch. 21)

Chapter 21: Reinforcement Learning

1. What is Reinforcement Learning?

- Reinforcement learning (RL) is learning from interaction with an environment.
- An agent takes actions, receives rewards, and learns a policy to maximize long-term reward.
- No labeled correct actions are given (unlike supervised learning).
- It is a trial-and-error approach to learning.

Key Idea: Learn to act so as to maximize the expected cumulative reward over time.

Example: Game playing, robot control, navigation, recommendation, resource management.

2. Main Components

Agent	Learns and selects actions.
Environment	Everything outside the agent that responds to actions.
State (s)	A representation of the current situation.
Action (a)	A choice the agent can make in a state.
Reward (r)	A scalar feedback signal from the environment.
Policy (π)	A mapping from states to actions. $\pi(a\|s)$ = probability of taking a in state s.
Value	Expected long-term return from a state (or state-action pair).

3. The Agent-Environment Loop



At each time step t:

1. Agent observes state s_t.
2. Agent selects an action a_t (based on policy).
3. Environment transitions to a new state s_{t+1} and gives a reward r_{t+1}.
4. Agent updates its knowledge and repeats.

4. Sequential Decision Making

- RL problems are sequential: actions affect future states and future rewards.
- The agent must consider long-term consequences, not just immediate reward.
- The environment is usually modeled as a Markov Decision Process (MDP).

Markov Property: The next state and reward depend only on the current state and action:

$$P(s_{t+1}, r_{t+1} \mid s_t, a_t) = P(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

(not on the full history)

5. Rewards, Return, and Discounting

Reward (r_t)

- Immediate scalar feedback from the environment.
- Can be positive, negative, or zero.
- Often sparse and delayed.

Return (G_t)

- Total discounted reward from time t onward:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- R_{t+k+1} is the reward received at time $t+k+1$.
- γ $(0 \le \gamma \le 1)$ is the discount factor.
- G_t measures the long-term value of a state or action.

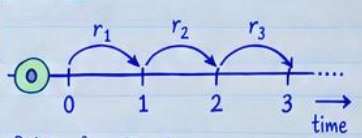
Discounting Intuition:

- $0 \le \gamma < 1$ makes future rewards count less.
- Encourages the agent to prefer sooner rewards.
- Helps with infinite horizons and ensures the sum converges (if rewards are bounded).
- Common values: $\gamma = 0.9,\ 0.95,\ 0.99$.

Why Discount Future Rewards?

- Sooner rewards are usually more valuable.
- Uncertain future (may never be received).
- Mathematical convenience: ensures the return is finite for an infinite horizon.

Example: $\gamma = 0.9$



Return from time 0:

$$G_0 = r_1 + 0.9\, r_2 + 0.9^2 r_3 + \dots$$

Rewards farther in the future contribute less to the return.

6. Policy and Behavior

- A policy π specifies what action to take in each state.
- Deterministic policy: $\pi(s) = a$ (a single action).
- Stochastic policy: $\pi(a|s) = P(a_t = a \mid s_t = s)$.
- The goal of RL is to find an optimal policy π^* that maximizes the expected return.

Optimal Policy:

$$\pi^* = \arg\max_{\pi} E_\pi[G_t]$$

7. Utility and Value Functions

- Value functions estimate the expected return:

$$V^\pi(s) = E_\pi[G_t \mid s_t = s] \quad \text{(state value)}$$

$$Q^\pi(s,a) = E_\pi[G_t \mid s_t = s, a_t = a] \quad \text{(state-action value)}$$

- $V^\pi(s)$ is the expected return starting from state s and following policy π.
- $Q^\pi(s,a)$ is the expected return starting from state s, taking action a, then following π.

Optimal Value Functions:

$$V^*(s) = \max_\pi V^\pi(s)$$

$$Q^*(s,a) = \max_\pi Q^\pi(s,a)$$

Relationship to Utility

- In RL, the value function (expected return) plays the role of a utility function.
- The agent prefers states with higher expected return.
- Optimal actions are those that lead to higher expected return.

Bellman Optimality Equation (intuition)

$$V^*(s) = \max_a \sum_{s',r} P(s', r \mid s, a)\left[r + \gamma V^*(s')\right]$$

$$Q^*(s,a) = \sum_{s',r} P(s', r \mid s, a)\left[r + \gamma \max_{a'} Q^*(s', a')\right]$$

8. Trial-and-Error Learning

- The agent does not know the environment's dynamics (transition probabilities or rewards).
- It must learn from experience through trial and error.
- The agent explores actions, observes rewards and next states, and updates value estimates or the policy based on the experience.

Key Idea: Learn what is good by trying actions and observing their long-term outcomes.

9. Exploration vs. Exploitation

- **Exploitation:** choose the action that currently appears best (highest estimated value).
- **Exploration:** try less certain actions to learn more about the environment.
- **Trade-off:** too much exploitation may miss better actions; too much exploration may waste time and reward.

Common Strategies:

- ϵ-greedy: with probability ϵ explore randomly.
- Softmax (Boltzmann) action selection.
- Upper Confidence Bounds (UCB).
- Thompson sampling (probabilistic exploration).

10. Key Takeaways

1. RL learns from interaction with an environment using rewards.
2. Goal: maximize expected cumulative (discounted) reward.
3. Key elements: agent, environment, state, action, reward, policy, value.
4. Learning is trial-and-error with delayed reward.
5. Return sums future rewards, with a discount factor γ.
6. Exploration is necessary to find good actions.
7. Value functions estimate how good states and actions are.
8. RL balances exploration and exploitation.

Chapter 21 : Reinforcement Learning

1. What is Reinforcement Learning (RL)?

- An agent learns by interacting with an environment, receiving rewards, and trying to maximize cumulative reward (utility).
- No model of the environment is given.
- Learns from experience (trial and error).
- Suitable for sequential decision making and delayed rewards.

Key Idea: Learn a policy that maps states to actions so as to maximize expected cumulative reward.

Example: Robot learning to walk, game playing, resource management, recommendation systems.

2. Formal RL Framework

State space (S)	:	Set of possible states
Action space (A)	:	Set of possible actions
Reward signal (R)	:	Immediate reward r_{t+1}
Policy (π)	:	Mapping from states to actions: $\pi(a \mid s)$
Return (G_t)	:	Cumulative discounted reward $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$
Goal	:	Find an optimal policy π^* that maximizes $E(G_t)$.

A solution is a policy that maximizes the expected cumulative discounted reward.

3. Learning from Experience

- Agent generates episodes by interacting with the environment.
- Uses observed transitions (s, a, r, s') to improve estimates of value/policy.
- Balances exploration (try new actions) and exploitation (use what is known).
- Does not require a model of the transition dynamics.

Example: Learn to play a game only from game experience, without knowing the rules or transition probabilities.

4. Function Approximation

- Needed when state space is very large or continuous.
- Represent value function or policy with parametric function (e.g., linear model, neural network).
- Generalizes knowledge across states.
- Updates parameters to reduce prediction error or improve policy.

Example: Approximate $V(s)$

$$V(s; w) = w^T \phi(s)$$

- $\phi(s)$: feature vector (e.g., state features)
- w: learned weights (parameters).

5. Handling Large State Spaces

- Explicitly representing a table for $V(s)$ or $Q(s, a)$ is infeasible.
- Use function approximation, state abstraction, or feature representation.
- Generalization allows learning about unseen states from similar states.
- Examples: tile coding, radial basis functions, neural networks.

Key Idea: Learn a compact representation that generalizes well across states, so the agent can perform well in unseen situations.

6. Generalization Across States

- Similar states should have similar values and lead to similar actions.
- Function approximation enables sharing of experience across many states.
- Helps the agent learn efficiently in large or continuous domains.
- Choice of features affects generalization and performance.

Example: In a grid world with many states, states with similar positions or features can share value estimates.

7. Relation to Dynamic Programming

- Dynamic programming (DP) assumes a known model $P(s' \mid s, a)$ and $R(s, a)$.
- RL learns from experience without a model.
- RL methods can be viewed as sample-based versions of DP (e.g., TD learning is a sample-based form of value iteration).
- Same fundamental ideas: value functions, Bellman equations, and policy improvement, but estimates are learned instead of computed from a model.

8. Methods for Learning Value/Policy

(a) Temporal-Difference (TD) Learning

- Learn value estimates from experience.
- Bootstrap from current estimates.
- Update toward sample-based target.

TD (state-value) update:

$$V(s) \leftarrow V(s) + \alpha \left[r + \gamma V(s') - V(s) \right]$$

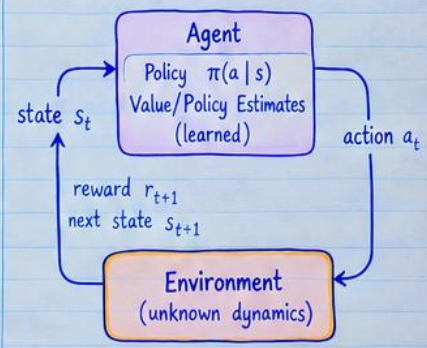
(b) Direct Utility Estimation (Monte Carlo)

- Learn value from complete episodes.
- Use average of observed returns.
- No bootstrapping (wait for final reward).

(c) Approximate Dynamic Programming (ADP)

- General framework for approximate solutions.
- Combines sampling, function approximation, and DP-style updates.
- Includes methods such as value iteration with function approximation and policy iteration.

9. Simple Diagram: Agent-Environment Loop



Agent learns a policy from the sequence of experience tuples (s, a, r, s') to maximize expected cumulative reward.

10. Methods Comparison

Method	What it learns	Target / Update	Pros	Cons	Typical use
Temporal-Difference (TD)	Value function (V or Q)	Bootstrapping: $r + \gamma V(s')$ (or Q)	Online, low variance, works well in large domains	Biased estimates, can be sensitive to step size	TD(0), SARSA, Q-learning
Direct Utility Estimation (Monte Carlo)	Value function	Sample average of returns G_t	Unbiased (with enough samples), simple	High variance, must wait for complete episodes	Episodic tasks, when episodes are short
Approximate Dynamic Programming (ADP)	Value or policy	DP-style updates with function approximation	Scales to large/continuous spaces, general framework	Approximation error, can be complex	Value iteration, policy iteration with function approx.

11. Key Takeaways

1. RL learns policies from experience to maximize cumulative reward.
2. Function approximation is essential for large or continuous state spaces.
3. Generalization allows knowledge transfer to unseen states.
4. RL is a sample-based alternative to dynamic programming.
5. Temporal-difference learning uses bootstrapping from current estimates.
6. Monte Carlo methods use complete returns without bootstrapping.
7. ADP unifies sampling, function approximation, and DP ideas.
8. These methods enable RL to work in realistic, large-scale problems.

> "Reinforcement learning learns to make good decisions by experience, even when the world is large, unknown, and uncertain."
> — AIMA (Ch. 21)

Chapter 21: Reinforcement Learning

1. Reinforcement Learning Framework

- Agent learns by interacting with an environment.
- No model of the environment is given.
- Receives reward signals (not explicit correct actions).
- Goal is to learn a policy that maximizes expected cumulative reward.

Key Idea: Learn from trial and error through experience while choosing actions.

2. Choosing Actions While Learning

- Must balance exploration (try new actions) and exploitation (use what has been learned).
- Unlike planning problems, the agent does not know the model, so it must learn values while acting.
- An action is selected at each state based on the current knowledge (e.g., estimated Q-values).

Challenge: Actions affect future states and future rewards, so short-term reward is not enough.

3. Exploitation vs. Exploration

Exploitation	Exploration
• Choose the best known action.	• Try less-known actions.
• Uses current knowledge to get high reward now.	• Helps discover actions that may lead to higher long-term reward.
• Risk: can miss better actions.	• Risk: lower immediate reward.

4. Exploration Strategies

(a) ε-Greedy Exploration

- With probability $1-\varepsilon$ choose the best known action.
- With probability ε choose a random action.
- Simple and widely used.

ε: exploration rate (e.g., 0.1)

(b) Decaying Exploration

- Start with larger ε and decrease over time.
- Explores a lot at the beginning, exploits more later.
- Common in practical applications.

(c) Softmax (Boltzmann) Exploration

- Choose actions probabilistically based on estimated values.

$$P(a \mid s) = \frac{e^{Q(s,a)/\tau}}{\sum_b e^{Q(s,b)/\tau}}$$

- τ (temperature) controls exploration:
  high $\tau \rightarrow$ more exploration,
  low $\tau \rightarrow$ more exploitation.

5. Exploration Function (General)

- An exploration function $f(n)$ returns a bonus for less tried actions.
- UCB (Upper Confidence Bound) is one example:

$$f(n) = c\sqrt{\frac{\ln N(s)}{n(s,a)}}$$

- $n(s,a)$: number of times action a taken in state s.
- $N(s)$: total number of times state s has been visited.
- c: controls the amount of exploration.

Encourages trying actions that have been tried less often.

6. Q-Values and Action-Value Function

- $Q(s,a)$ is the expected cumulative reward obtained by taking action a in state s and then following a policy π.

$$Q^{\pi}(s,a) = E\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s,\ A_t = a,\ \pi\right]$$

- π is the policy used after the first action.
- $\gamma \in [0,1)$ is the discount factor.
- Q-values combine immediate and future rewards.
- The optimal action-value function $Q^*(s,a)$ is the maximum expected return over all policies.

If we learn $Q^*(s,a)$, we can act optimally by choosing the action with highest Q-value.

7. Q-Learning

- Model-free reinforcement learning algorithm.
- Learns the optimal action-value function $Q^*(s,a)$ without knowing the environment model.
- Can be combined with exploration strategies (e.g., ε-greedy).
- Converges to $Q^*(s,a)$ under suitable conditions (e.g., sufficient exploration, decreasing learning rate).

8. Q-Learning Update Rule

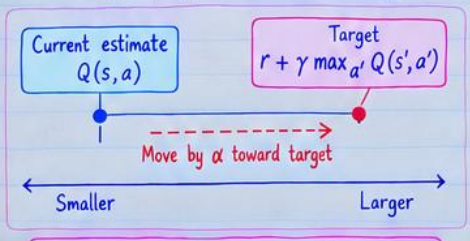
Update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

- r: reward received after taking action a in state s.
- s': next state.
- α: learning rate $(0 < \alpha \leq 1)$.
- γ: discount factor.
- $\max_{a'} Q(s',a')$: estimate of optimal future value from next state.

9. Intuition Behind the Update Rule

- The agent takes action a in state s, observes reward r and next state s'.
- The target value is the observed reward plus the best estimated value of the next state.
- The Q-value moves a small step (α) toward the target (temporal-difference learning).



Repeated updates cause $Q(s,a)$ to converge to the true optimal value $Q^*(s,a)$.

10. Policy and Improvement

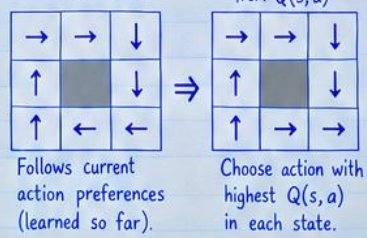
- A policy $\pi(s)$ maps each state to an action.
- After learning Q-values, we can derive an improved policy:

$$\pi(s) = \arg\max_a Q(s,a)$$

Policy Improvement (Greedy):

1. Keep $Q(s,a)$ learned from interaction.
2. For each state s, choose the action with highest $Q(s,a)$.
3. This gives a better policy, which can be used to generate more experience.

Example: Policy Improvement in a Grid



11. Summary and Key Takeaways

1. RL learns by acting and receiving rewards (trial and error).
2. Must balance exploration and exploitation.
3. Exploration strategies: ε-greedy, decaying ε, softmax, UCB.
4. Q-values estimate expected cumulative reward.
5. Q-learning updates $Q(s,a)$ using the temporal-difference rule.
6. Learn $Q^*(s,a)$ and act greedily to obtain an optimal policy.
7. With sufficient exploration, Q-learning converges to optimal values.

"Reinforcement learning enables an agent to learn from experience and improve through interaction." — AIMA (Ch. 21)

Chapter 21 : Reinforcement Learning

1. What is Reinforcement Learning?

- Agent learns by interacting with an environment to maximize cumulative reward.
- No explicit model of the environment is needed (unlike planning).
- Learns from experience through trial and error.

Key Idea: Learn a policy that maximizes the expected cumulative reward.

Example: Robot learning to walk, game playing, autonomous driving.

2. Important Formulas

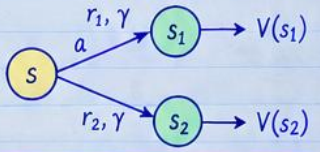
- Return: $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ $\quad 0 \le \gamma < 1$ (discount factor); $\gamma = 0$: myopic; $\gamma \to 1$: long-term
- State-value function: $V^\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$
- Action-value function: $Q^\pi(s,a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a]$
- Bellman expectation equation (for V^π): $V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} P(s',r|s,a) \left[r + \gamma V^\pi(s') \right]$
- Optimality (Bellman optimality): $V^*(s) = \max_a \sum_{s',r} P(s',r|s,a) \left[r + \gamma V^*(s') \right]$

$$Q^*(s,a) = \sum_{s',r} P(s',r|s,a) \left[r + \gamma \max_{a'} Q^*(s',a') \right]$$

3. Bellman Idea

- A value can be expressed in terms of the values of successor states.
- Dynamic programming and many RL algorithms are based on the Bellman equation.

Bellman intuition:
The value of a state equals the immediate reward plus the discounted value of what happens next.



Value of s depends on immediate rewards and values of successor states.

4. Utility vs. Reward vs. Return

Term	Meaning
Reward (R)	• Immediate feedback from environment. • Can be positive, negative or zero. • Defines the task.
Return (G)	• Cumulative discounted sum of rewards from time t. $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$
Utility / Value (V or Q)	• Expected return from a state (or state-action pair) under a policy. • Represents long-term desirability.

5. Passive vs. Active Reinforcement Learning

Passive RL (Policy Evaluation)

- Policy is fixed (given).
- Learn the value function $V^\pi(s)$ or $Q^\pi(s,a)$.
- Exploration not required (must follow policy).
- Example algorithm: Monte Carlo evaluation, Temporal-Difference (TD) learning.

Active RL (Control)

- Learn the optimal policy while interacting.
- Must balance exploration and exploitation.
- Learn either:
  - Value function and derive policy
  - or directly learn a policy.
- Example algorithms: Q-learning, SARSA, Policy iteration, Value iteration, Policy gradients.

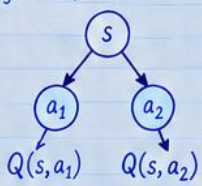
Control = evaluate + improve → leads to optimal policy.

6. Policy and Q-Values

- Policy (π): mapping from states to actions. $\pi(a|s)$ = probability of taking action a in state s.
- State-value: $V^\pi(s)$ = expected return starting from s.
- Action-value: $Q^\pi(s,a)$ = expected return starting from s, taking action a.
- Optimal policy: $\pi^*(s) = \arg\max_a Q^*(s,a)$

Key relation:

$$V^\pi(s) = \sum_a \pi(a|s)\, Q^\pi(s,a)$$



Choose action with highest Q-value (greedy policy).

7. Common Algorithms (Summary)

Algorithm	Type	Key Idea	Model?
Monte Carlo (MC)	Passive	Learn from complete episodes using sample returns.	No
Temporal Difference (TD, e.g. TD(0))	Passive	Bootstraps from current estimate (no need to wait).	No
SARSA	Active (on-policy)	Updates using action actually taken (on-policy).	No
Q-learning	Active (off-policy)	Learns optimal Q* using max over next actions.	No
Policy Iteration	Planning (model-based)	Alternate policy evaluation and policy improvement.	Yes
Value Iteration	Planning (model-based)	Directly computes optimal values using Bellman backup.	Yes
Policy Gradient	Active	Learn policy parameters directly via gradient ascent.	No

8. Common Pitfalls

- Confusing reward with utility/value.
- Ignoring the discount factor (γ).
- Not exploring enough (gets stuck in local optimum).
- Non-stationary policies when using function approximation.
- Assuming immediate rewards capture long-term goals.
- Overestimating Q-values (bias in off-policy learning).
- Function approximation instability (e.g. with deep RL).
- Ignoring the exploration–exploitation trade-off.

9. Likely Exam Questions

1. Define reinforcement learning and its key components.
2. Derive the Bellman expectation and optimality equations.
3. Explain the difference between utility, reward and return.
4. Distinguish between passive and active RL.
5. Describe and compare Monte Carlo and Temporal Difference learning.
6. Derive the update rule for Q-learning (or SARSA).
7. Explain the exploration–exploitation dilemma and common strategies (e.g. ε-greedy).
8. Discuss advantages and limitations of model-free vs. model-based RL.

10. Applications

- Game playing (e.g. Atari, Go, chess).
- Robotics and control (e.g. robot walking).
- Autonomous driving.
- Resource management and scheduling.
- Recommendation systems.
- Healthcare and treatment planning.
- Finance and automated trading.
- Dialogue and interactive systems.
- Real-world sequential decision making under uncertainty.

11. Key Takeaways (Final Revision)

1. RL learns a policy by maximizing expected cumulative reward through interaction.
2. The Bellman equation expresses value in terms of immediate reward and future value.
3. Value functions (V and Q) represent long-term expected return.
4. Passive RL evaluates a fixed policy; active RL learns the optimal policy.
5. Key algorithms: MC, TD, SARSA, Q-learning, policy iteration, value iteration, policy gradients.
6. Explorations and exploitation are crucial for good learning.
7. RL has wide real-world applications but can be challenging due to stability and sample efficiency issues.

"Reinforcement learning turns experience into intelligence by learning what to do, not just what to think." — AIMA (Ch.21)